

到。随后，**重排策略**通过调整请求的处理顺序，优先处理那些能够更多利用缓存数据的请求，以减少重新计算的需求。这样的请求在队列中享有较高的优先级，从而优化了资源使用。最后，系统还引入了**动态推测流水线策略**，通过并行 KV 张量检索和模型推理的步骤，有效减少了端到端延迟。

综上，通过结合上述**输入 Prompt 压缩**与**KV-cache 机制**，RAG 框架可以在保持高性能的同时，显著提升其效率。这不仅有助于在资源受限的环境中部署模型，还可以提高模型在实际应用中的响应速度。

6.5 实践与应用

通过引入外部知识，RAG 可以有效的缓解大语言模型的幻觉现象，拓展了大语言模型的知识边界。其优越的性能使引起了广泛关注，成为炙手可热的前沿技术，并在众多应用场景中落地。在本节中，我们将探讨搭建简单 RAG 系统的方法，以及 RAG 的两类典型应用。

6.5.1 搭建简单 RAG 系统

为了助力开发者们高效且便捷地构建 RAG 系统，当前已有诸多成熟开源框架可供选择，其中最具代表性的便是 LangChain⁸与 LlamaIndex⁹。这些框架提供了一系列完备的工具与接口，使得开发者们能够轻松地将 RAG 系统集成到他们的应用中，例如聊天机器人、智能体（Agent）等。接下来，我们将首先简要概述这两个框架的特色及其核心功能，然后讲解如何利用 LangChain 来搭建一个简单的 RAG 系统。

⁸<https://www.langchain.com>

⁹<https://www.llamaindex.ai>

1. LangChain 与 LlamaIndex

(1) LangChain

LangChain 旨在简化利用大语言模型开发应用程序的整个过程。它提供了一系列模块化的组件，帮助开发者部署基于大语言模型的应用，其中就包括 RAG 框架的构建。LangChain 主要包含六大模块：**Model IO**、**Retrieval**、**Chains**、**Memory**、**Agents** 和 **Callbacks**。其中 Model IO 模块包含了各种大模型的接口以及 Prompt 设计组件，Retrieval 模块包含了构建 RAG 系统所需要的核心组件，包括**文档加载**、**文本分割**、**向量构建**、**索引生成以及向量检索**等，还提供了非结构化数据库的接口。而 Chains 模块可以将各个模块链接在一起逐个执行，Memory 模块则可以存储对话过程中的数据。此外，Agent 模块可以利用大语言模型来自动决定执行哪些操作，Callback 模块则可以帮助开发者干预和监控各个阶段。总体而言，**LangChain 提供了一个较为全面的模块支持**，帮助开发者们轻松便捷地构建自己的 RAG 应用框架。

(2) LlamaIndex

与 LangChain 相比，LlamaIndex 更加专注于**数据索引与检索**的部分。这一特性使得开发者能够**迅速构建高效的检索系统**。LlamaIndex 具备从多种数据源（如 API、PDF 文件、SQL 数据库等）中提取数据的能力，并提供了一系列高效的工具来对这些数据进行向量化处理和索引构建。在数据查询方面，LlamaIndex 同样提供了高效的检索机制。此外，在获取到上下文信息后，LlamaIndex 还支持对这些信息进行过滤、重新排序等精细化操作。值得一提的是，LlamaIndex 框架还能够与 LangChain 框架相结合，从而实现更加多样化的功能。总体而言，**LlamaIndex 侧重于索引与检索，在查询效率上的表现更为突出**，非常适用于在大数据量的场景下构建更为高效的 RAG 系统。

2. 基于 LangChain 搭建简单 RAG 系统

本小节将以 LangChain 框架为例，参考其官方文档¹⁰，演示如何快速搭建一套简单的 RAG 系统。

1) 安装与配置：首先，需要在环境中安装 LangChain 框架及其依赖项。

```
# 安装LangChain框架及其依赖项
!pip install langchain langchain_community langchain_chroma
```

2) 数据准备与索引构建：接下来，我们需要准备数据并构建索引。LangChain 的 DocumentLoaders 中提供了种类丰富的文档加载器，例如，我们可以使用 WebBaseLoader 从网页中加载内容并将其解析为文本。

```
from langchain_community.document_loaders import WebBaseLoader
# 使用WebBaseLoader加载网页内容：
loader = WebBaseLoader("https://example.com/page")
docs = loader.load()
```

加载完成后，由于加载的文档可能过长，不适合模型的上下文窗口，需要将文档分割成合适的大小。LangChain 提供了 TextSplitter 组件来实现文档分割。

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
# 使用TextSplitter将长文档分割成更小的块，其中chunk_size表示分割文档的长度，
# chunk_overlap表示分割文档间的重叠长度
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
splits = text_splitter.split_documents(docs)
```

接下来我们需要对分割后的文本块进行索引化，以便后续进行检索。这里我们可以调用 Chroma 向量存储模块和 OpenAIEmbeddings 模型来存储和编码文档。

¹⁰<https://python.langchain.com/v0.2/docs/tutorials/rag/>

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
# 使用向量存储(如Chroma)和嵌入模型来编码和存储分割后的文档
vectorstore = Chroma.from_documents(documents=splits, embedding=OpenAIEmbeddings
    ())
```

3) RAG 系统构建：在构建好知识源之后，接下来可以开始构建一个基础的 RAG 系统。该系统包括检索器与生成器两部分，具体工作流程如下：对于用户输入的问题，检索器首先搜索与该问题相关的文档，接着将检索到的文档与初始问题一起传递给生成器，即大语言模型，最后将模型生成的答案返回给用户。

首先进行检索器的构建，这里我们可以基于 `VectorStoreRetriever` 构建一个 `Retriever` 对象，利用向量相似性进行检索。

```
# 创建检索器
retriever = vectorstore.as_retriever()
```

接下来是生成器部分的构建，这里我们可以使用 `ChatOpenAI` 系统模型作为生成器。在这一步骤中，需要设置 OpenAI 的 API 密钥，并指定要使用的具体模型型号。例如，我们可以选择使用 `gpt-3.5-turbo-0125` 模型。

```
import os
os.environ["OPENAI_API_KEY"] = 'xxx'
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-turbo-0125")
```

随后是输入 Prompt 的设置，LangChain 的 Prompt Hub 中提供了多种预设的 Prompt 模板，适用于不同的任务和场景。这里我们选择一个适用于 RAG 任务的 Prompt。

```
from langchain import hub

# 设置提示模板

prompt = hub.pull("rlm/rag-prompt")
```

最后我们需要整合检索与生成,这里可以使用 LangChain 表达式语言(LangChain Execution Language, LCEL) 来方便快捷地构建一个链,将检索到的文档、构建的输入 Prompt 以及模型的输出组合起来。

```
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

# 使用LCEL构建RAG链
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

# 定义文档格式化函数
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

# 使用RAG链回答问题
response = rag_chain.invoke("What is Task Decomposition?")
print(response)
```

通过以上步骤,我们可以方便快捷地使用 LangChain 迅速搭建一个基础 RAG 系统。LangChain 提供了一系列强大的工具和组件,使得构建和整合检索与生成过程变得简单而高效。

6.5.2 RAG 的典型应用

本小节将介绍 RAG 系统的两个典型应用案例：(1) 智能体 (Agent)；(2) 垂域多模态模型增强。

1. 智能体 (Agent)

RAG 在 Agent 系统中扮演着重要角色。在 Agent 系统主动规划和调用各种工具的过程中，需要检索并整合多样化的信息资源，以更精确地满足用户的需求。RAG 通过提供所需的信息支持，助力 Agent 在处理复杂问题时展现出更好的性能。图 6.27 展示了一个经典的 Agent 框架 [50]，该框架主要由四大部分组成：配置 (Profile)、记忆 (Memory)、计划 (Planning) 和行动 (Action)。其中记忆模块、计划模块与行动模块均融入了 RAG 技术，以提升整体性能。

具体而言，(1) 配置模块通过设定基本信息来定义 Agent 的角色，这些信息可以包括 Agent 的年龄、性别、职业等基本属性，以及反映其个性和社交关系的信息；(2) 记忆模块存储从环境中学习到的知识以及历史信息，支持记忆检索、记忆更新和记忆反思等操作，允许 Agent 不断获取、积累和利用知识。在这一模块中，RAG 通过检索相关信息来辅助记忆的读取和更新；(3) 计划模块赋予 Agent 将复杂任务分解为简单的子任务的能力，并根据记忆和行动反馈不断调整。RAG 在此模块中通过提供相关的信息，帮助 Agent 更合理有效地规划任务；(4) 行动模块则负责将 Agent 的计划转化为具体的行动，包括网页检索、工具调用以及多模态输出等，能够对环境或 Agent 自身状态产生影响，或触发新的行动链。在这一模块中，RAG 通过检索相关信息来辅助 Agent 的决策和行动执行。通过这种模块化且集成 RAG 技术的方法，Agent 能够更加高效和智能地响应用户需求，展现出更加卓越的性能。

我们以“我现在在澳大利亚，我想去抱考拉，应该怎么办呢？”这一具体问题

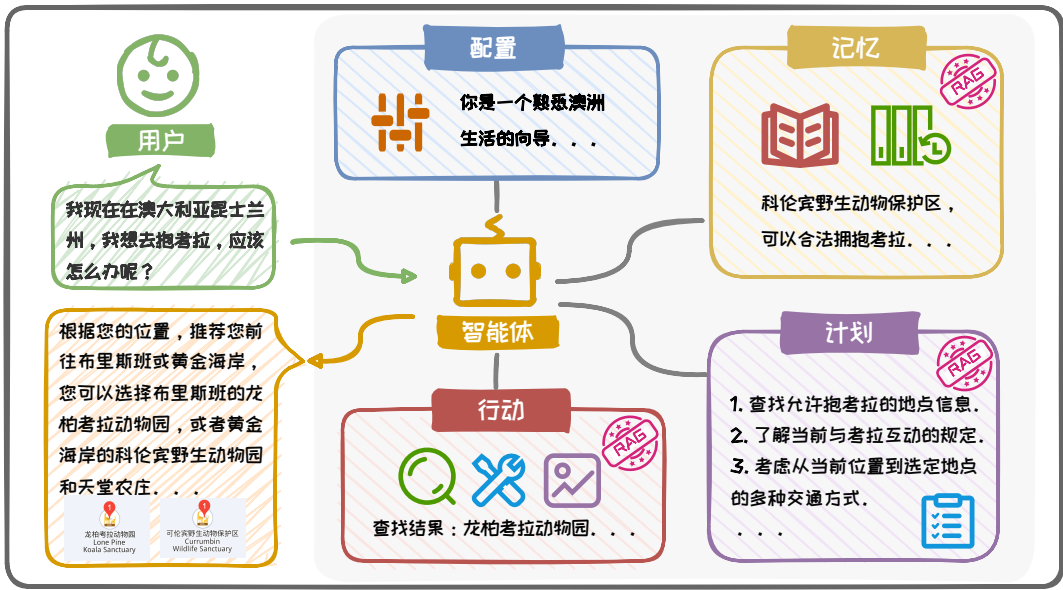


图 6.27: Agent 框架流程示意图。

为例，来展示 Agent 处理问题的流程。在这个例子中，用户提出了一个明确的需求。Agent 将通过以下步骤来完成任务：

- **角色配置：**首先，Agent 利用**配置模块**进行初始化，设定其角色为专业的旅游顾问，明确其任务目标是协助用户实现与考拉亲密接触的愿望；
- **任务规划：**针对用户的需求，**计划模块**规划如何帮助用户实现“抱考拉”的愿望。例如确定当前位置附近存在考拉的地点、了解当地关于与野生动物互动的法律规定、以及前往目的地的方式等。在这一阶段，**RAG** 通过提供相关的旅游景点和法律法规信息，使规划更加精准；
- **信息检索：****记忆模块**首先在记忆中检索已有的相关信息。如果记忆中不包含所有必要的信息，**行动模块**将激活并调用工具从外部知识源（如旅游网站等）中进行检索。在这一过程中，**RAG** 确保以最高效率检索到与考拉互动最相关的信息；
- **信息整合与决策：****计划模块**将利用上述检索到的信息制定最佳行动方案。这可能涉及推荐特定的地点、提供出行建议和强调安全须知。**RAG** 在此环节中

整合信息，辅助 Agent 做出明智的决策；

- **信息输出：**最后，**行动模块**将 Agent 确定的行动方案以易于理解的方式输出给用户，包括详细的路线规划以及相关地点的描述、图像等信息。

通过这个例子，我们可以清晰地看到 Agent 框架中的各个模块如何协同工作来完成一个具体的预测任务，其中 RAG 的作用在于快速检索信息并整合知识，为用户提供一个全面而精确的解决方案。

2. 多模态垂直领域应用

在前面的章节中，我们主要聚焦于文本领域的 RAG 系统，而今，在诸多涉及到多模态数据的垂直领域中，RAG 也展现出了广阔的应用前景。例如，在医疗领域中，多模态数据十分普遍，包括 X 光片、MRI、CT 扫描等影像资料，病历、生理监测数据等文本资料。这些数据不仅来源广泛，而且彼此之间存在着复杂的相互联系。因此，在应对这些任务时，RAG 系统必须具备融合与洞察不同模态数据的能力，以确保其精准高效地发挥作用。

目前，已经出现了一些多模态垂直领域的 RAG 应用，例如 Ossowski 等人提出的医学领域多模态检索框架 [38]。图 6.28 展示了该框架的整个工作流程。此处以**给定一张 X 光照片，询问该照片所对应的可能症状**这一任务为例进行说明。该框架首先**提取图像与文本的特征表示**，深入理解图像内容 with 问题语义，为检索模块提供丰富的特征向量。随后，该框架精心设计了一个**多模态检索模块**，利用这些特征向量在医学知识库中进行精准检索，从而获取与输入问题最为相关的信息。这些信息可能包括医学案例、症状描述、潜在病因及治疗方案等，它们以文本和图像的多种形式呈现。最终，Prompt 构建模块将这些信息进行整合，辅助模型生成准确的回答。在本例中，模型生成的答案是“心肌肥大”。

除了医疗领域，RAG 在其他垂直领域，如金融 [8]、生物学 [53] 中也展现了其在处理专业信息上的强大能力，显著提升了专业人士的决策效率。在各类任务中，

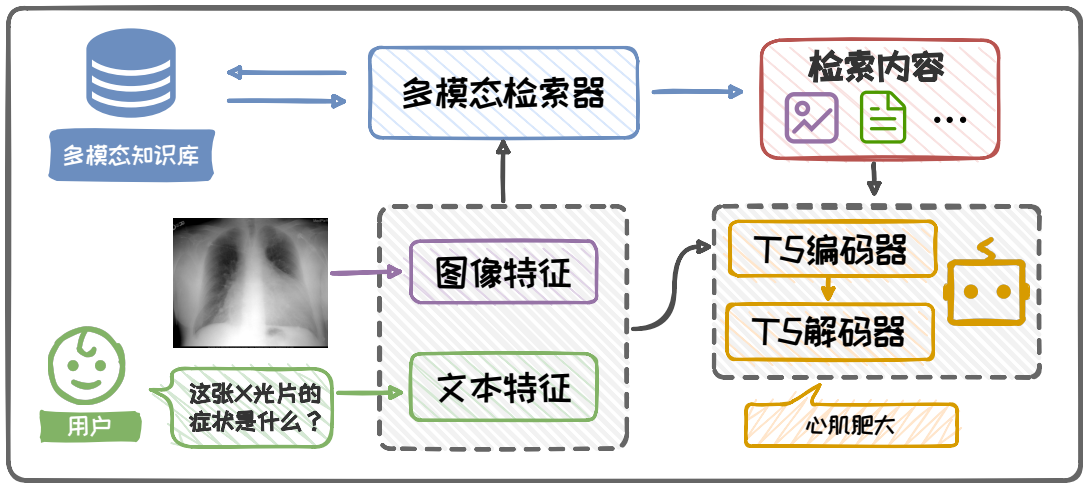


图 6.28: 多模态垂域 RAG 框架示意图。

除了图片信息 [10], RAG 在音频 [7]、视频 [52] 等其他多模态场景中也同样有着出色的表现。随着技术的进步和数据资源的丰富, 我们期待未来 RAG 能够在更多垂直领域、更多数据模态中发挥关键作用。

参考文献

- [1] Josh Achiam et al. "Gpt-4 technical report". In: *arXiv preprint arXiv:2303.08774* (2023).
- [2] Akiko Aizawa. "An information-theoretic perspective of tf-idf measures". In: *IPM*. 2003.
- [3] Akari Asai et al. "Self-rag: Learning to retrieve, generate, and critique through self-reflection". In: *arXiv preprint arXiv:2310.11511* (2023).
- [4] Jon Louis Bentley. "Multidimensional binary search trees used for associative searching". In: *Communications of the ACM* (1975).
- [5] Sebastian Borgeaud et al. "Improving language models by retrieving from trillions of tokens". In: *ICML*. 2022.
- [6] Sebastian Borgeaud et al. "Improving language models by retrieving from trillions of tokens". In: *ICML*. 2022.

- [7] David M Chan et al. “Using external off-policy speech-to-text mappings in contextual end-to-end automated speech recognition”. In: *arXiv preprint arXiv:2301.02736* (2023).
- [8] Jian Chen et al. “FinTextQA: A Dataset for Long-form Financial Question Answering”. In: *arXiv preprint arXiv:2405.09980* (2024).
- [9] Jiangui Chen et al. “A Unified Generative Retriever for Knowledge-Intensive Language Tasks via Prompt Learning”. In: *SIGIR*. 2023.
- [10] Wenhui Chen et al. “Re-imagen: Retrieval-augmented text-to-image generator”. In: *arXiv preprint arXiv:2209.14491* (2022).
- [11] Mayur Datar et al. “Locality-sensitive hashing scheme based on p-stable distributions”. In: *SoCG*. 2004.
- [12] Hervé Déjean, Stéphane Clinchant, and Thibault Formal. “A Thorough Comparison of Cross-Encoders and LLMs for Reranking SPLADE”. In: *arXiv preprint arXiv:2403.10407* (2024).
- [13] Mohamad Dolatshah, Ali Hadian, and Behrouz Minaei-Bidgoli. “Ball*-tree: Efficient spatial indexing for constrained nearest-neighbor search in metric spaces”. In: *arXiv preprint arXiv:1511.00628* (2015).
- [14] Paulo Finardi et al. “The Chronicles of RAG: The Retriever, the Chunk and the Generator”. In: *arXiv preprint arXiv:2401.07883* (2024).
- [15] Tiezheng Ge et al. “Optimized product quantization for approximate nearest neighbor search”. In: *CVPR*. 2013.
- [16] Samuel Humeau et al. “Poly-encoders: Transformer architectures and pre-training strategies for fast and accurate multi-sentence scoring”. In: *arXiv preprint arXiv:1905.01969* (2019).
- [17] Gautier Izacard et al. “Atlas: Few-shot learning with retrieval augmented language models”. In: *Journal of Machine Learning Research* (2023).
- [18] Herve Jegou, Matthijs Douze, and Cordelia Schmid. “Product quantization for nearest neighbor search”. In: *IEEE transactions on pattern analysis and machine intelligence* 33.1 (2010), pp. 117–128.
- [19] Hervé Jégou et al. “Searching in one billion vectors: re-rank with source coding”. In: *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2011, pp. 861–864.

- [20] Huiqiang Jiang et al. “Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression”. In: *arXiv preprint arXiv:2310.06839* (2023).
- [21] Chao Jin et al. “RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation”. In: *arXiv preprint arXiv:2404.12457* (2024).
- [22] Nikhil Kandpal et al. “Large language models struggle to learn long-tail knowledge”. In: *ICML*. 2023.
- [23] Vladimir Karpukhin et al. “Dense passage retrieval for open-domain question answering”. In: *arXiv preprint arXiv:2004.04906* (2020).
- [24] Omar Khattab and Matei Zaharia. “Colbert: Efficient and effective passage search via contextualized late interaction over bert”. In: *SIGIR*. 2020.
- [25] Omar Khattab et al. “Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp”. In: *arXiv preprint arXiv:2212.14024* (2022).
- [26] Gangwoo Kim et al. “Tree of clarifications: Answering ambiguous questions with retrieval-augmented large language models”. In: *EMNLP*. 2023.
- [27] Mike Lewis et al. “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension”. In: *arXiv preprint arXiv:1910.13461* (2019).
- [28] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *NeurIPS*. 2020.
- [29] Xiaoxi Li et al. “From matching to generation: A survey on generative information retrieval”. In: *arXiv preprint arXiv:2404.14851* (2024).
- [30] Yuxin Liang et al. “Learning to trust your feelings: Leveraging self-awareness in llms for hallucination mitigation”. In: *arXiv preprint arXiv:2401.15449* (2024).
- [31] Yu A Malkov and Dmitry A Yashunin. “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs”. In: *IEEE transactions on pattern analysis and machine intelligence* (2018).
- [32] Yury Malkov et al. “Approximate nearest neighbor algorithm based on navigable small world graphs”. In: *Information Systems* 45 (2014), pp. 61–68.
- [33] Alex Mallen et al. “When not to trust language models: Investigating effectiveness of parametric and non-parametric memories”. In: *ACL*. 2023.

- [34] Potsawee Manakul, Adian Liusie, and Mark JF Gales. “Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models”. In: *EMNLP*. 2023.
- [35] Yuren Mao et al. “FIT-RAG: Black-Box RAG with Factual Information and Token Reduction”. In: *ACM Transactions on Information Systems* (2024).
- [36] Kevin Meng et al. “Locating and editing factual associations in GPT”. In: *NeurIPS*. 2022.
- [37] Stanislav Morozov and Artem Babenko. “Non-metric similarity graphs for maximum inner product search”. In: *Advances in Neural Information Processing Systems* 31 (2018).
- [38] Timothy Ossowski and Junjie Hu. “Multimodal prompt retrieval for generative visual question answering”. In: *ACL*. 2023.
- [39] James Jie Pan, Jianguo Wang, and Guoliang Li. “Survey of vector database management systems”. In: *arXiv preprint arXiv:2310.14021* (2023).
- [40] Ella Rabinovich et al. “Predicting Question-Answering Performance of Large Language Models through Semantic Consistency”. In: *arXiv preprint arXiv:2311.01152* (2023).
- [41] Colin Raffel et al. “Exploring the limits of transfer learning with a unified text-to-text transformer”. In: *The Journal of Machine Learning Research* (2020).
- [42] Ori Ram et al. “In-context retrieval-augmented language models”. In: *Transactions of the Association for Computational Linguistics* (2023).
- [43] Stephen Robertson, Hugo Zaragoza, et al. “The probabilistic relevance framework: BM25 and beyond”. In: *Foundations and Trends® in Information Retrieval* (2009).
- [44] Ferdinand Schlatt, Maik Fröbe, and Matthias Hagen. “Investigating the Effects of Sparse Attention on Cross-Encoders”. In: *ECIR*. 2024.
- [45] Weijia Shi et al. “Replug: Retrieval-augmented black-box language models”. In: *arXiv preprint arXiv:2301.12652* (2023).
- [46] Hanshi Sun et al. “Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding”. In: *arXiv preprint arXiv:2404.11912* (2024).
- [47] Weiwei Sun et al. “Is chatgpt good at search? investigating large language models as re-ranking agent”. In: *arXiv preprint arXiv:2304.09542* (2023).
- [48] Yi Tay et al. “Transformer memory as a differentiable search index”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 21831–21843.

- [49] Hugo Touvron et al. “Llama 2: Open foundation and fine-tuned chat models”. In: *arXiv preprint arXiv:2307.09288* (2023).
- [50] Lei Wang et al. “A survey on large language model based autonomous agents”. In: *Frontiers of Computer Science* 18 (2024), p. 186345.
- [51] Yujing Wang et al. “A Neural Corpus Indexer for Document Retrieval”. In: *ArXiv abs/2206.02743* (2022). URL: <https://api.semanticscholar.org/CorpusID:249395549>.
- [52] Zhenhailong Wang et al. “Language models with image descriptors are strong few-shot video-language learners”. In: *NeurIPS*. 2022.
- [53] Zichao Wang et al. “Retrieval-based controllable molecule generation”. In: *ICLR*. 2022.
- [54] Jason Wei et al. “Chain of Thought Prompting Elicits Reasoning in Large Language Models”. In: *NeurIPS*. 2022.
- [55] David H Wolpert and William G Macready. “No free lunch theorems for optimization”. In: *IEEE transactions on evolutionary computation* 1 (1997), pp. 67–82.
- [56] Mingrui Wu and Sheng Cao. “LLM-Augmented Retrieval: Enhancing Retrieval Models Through Language Models and Doc-Level Embedding”. In: *arXiv preprint arXiv:2404.05825* (2024).
- [57] Haoyan Yang et al. “Prca: Fitting black-box large language models for retrieval question answering via pluggable reward-driven contextual adapter”. In: *EMNLP*. 2023.
- [58] Wenhao Yu et al. “Generate rather than retrieve: Large language models are strong context generators”. In: *ICLR*. 2023.
- [59] Wenhao Yu et al. “Improving Language Models via Plug-and-Play Retrieval Feedback”. In: *arXiv preprint arXiv:2305.14002* (2023).
- [60] Zichun Yu et al. “Augmentation-Adapted Retriever Improves Generalization of Language Models as Generic Plug-In”. In: *arXiv preprint arXiv:2305.17331* (2023).
- [61] Yujia Zhou et al. “DynamicRetriever: A Pre-training Model-based IR System with Neither Sparse nor Dense Index”. In: *ArXiv abs/2203.00537* (2022). URL: <https://api.semanticscholar.org/CorpusID:247187834>.